National Blockchain Voting System — Architecture & Design

Scale target: National election, tens of millions of eligible voters, single election-day peak load window
Deployment model: Hybrid — permissioned blockchain nodes on-premises at each authority, supporting services in cloud **Recommended blockchain platform:** Hyperledger Fabric

1. Why Hyperledger Fabric

For a national election, the blockchain's job is not "decentralization for its own sake" — it's giving mutually distrustful parties (electoral commission, political parties, judiciary, international observers) a shared, tamper-evident record without any one of them controlling it. Fabric fits this better than a public chain or a from-scratch ledger for concrete reasons:

Membership Service Providers (MSPs) map directly onto real-world organizations. Each stakeholder (Electoral Commission, each major party, Judiciary, National Audit Office) becomes a distinct **Fabric Organization**, each running its own peers, each with cryptographic identity issued by its own CA. No single org can act unilaterally.
Channels allow logical/physical partitioning — e.g., one channel per state/province — so the ledger scales horizontally instead of one monolithic chain carrying all national traffic.
Chaincode (smart contracts) in Go/Java/Node.js enforce ballot validation, spent-token checks, and tallying logic as code every peer runs identically — no custom consensus engine to build from scratch.
Pluggable ordering service — Raft (crash fault tolerant) is standard; Fabric's BFT ordering service (available from v3.0) is worth using here since we should tolerate not just crashes but malicious orderers, given the stakes.
Mature, audited, widely deployed in regulated industries (finance, healthcare) — meaningfully lowers the "unproven cryptography in production" risk for something this high-stakes.

Trade-off to be transparent about: Fabric is not a silver bullet. It gives you tamper-evident, multi-party-witnessed storage. It does **not** give you voter anonymity, coercion-resistance, or eligibility checking — those remain the job of the identity, casting, and crypto layers designed on top of it (Sections 5–7).

2. Consortium & network topology

Organization	Role	Node presence
Electoral Commission	Chain admin, identity anchor, orders peers	On-prem, primary data centers
Each registered political party (observer status)	Peer + orderer node, read/verify only	On-prem, party-operated hardware

Organization	Role	Node presence
Judiciary / Constitutional Court	Independent audit node	On-prem, judiciary infrastructure
National Audit Office / civil society coalition	Independent audit node	On-prem or co-located
International observers (e.g., UN/EU mission)	Read-only peer, verification only	On-prem or remote VPN-connected

Each organization holds **one share of the threshold decryption key** (Section 7) and **one vote in ordering-service consensus**. No result can be decrypted, and no ledger entry finalized, without a quorum of independent parties agreeing — this is the core trust property.

Channel sharding by province/state: each region gets its own Fabric channel, so peers only replicate the ballots relevant to their jurisdiction, and no single channel needs to handle the entire country's write throughput. A national aggregation service (Section 4) reads finalized per-channel tallies and combines them.

Scaling beyond a handful of provinces: one channel per province works cleanly at single-digit-to-dozens of jurisdictions. If the country has 50+ states or 100+ districts, flat province-level channels start to strain channel-management overhead (each channel carries its own gossip network, MSP config, and endorsement policy). At that scale, the design should shift to:

Hierarchical aggregation: district-level channels roll up into state-level aggregation nodes, which roll up into the national aggregator — a tree rather than a flat fan-out, so no single layer has to reconcile hundreds of channels at once.
Multiple ordering clusters: rather than one ordering service for the whole country, run regional ordering clusters (each still BFT, each still spanning the same consortium orgs) so ordering throughput scales with region count instead of bottlenecking centrally.
Partitioned tally services: the Tally Service and Threshold Decryption Coordinator should be horizontally partitioned by region too, each running its own decryption ceremony, with the National Aggregator only ever combining already-finalized, already-verifiable regional results.

This is a "when you outgrow it" note rather than a current gap — worth deciding the jurisdiction count early since it determines whether to build the flat or hierarchical version from day one.

3. Full component architecture

Layer	Component	Responsibility	Suggested stack
Client	Voter Web/Mobile App	Local ballot encryption, credential handling, offline-first UX	React Native (mobile) / React + WASM crypto (web)

Layer	Component	Responsibility	Suggested stack
Client	Poll-site Kiosk App	Assisted/in-person voting terminals	Electron or native, same crypto core as web app
Edge	API Gateway / WAF	TLS termination, rate limiting, DDoS mitigation, request routing	Kong / AWS API Gateway + Cloudflare or national CDN equivalent
Identity	Identity & Eligibility Service	Auth against national ID, blind-signs tokens, marks issuance	Go/Java microservice, integrates with national eID via SAML/OIDC
Identity	Voter Roll Database	Source of truth for eligibility (off-chain, encrypted at rest)	PostgreSQL, encrypted, replicated across regions
Identity	HSM Cluster (issuing keys)	Holds blind-signature/token-issuing keys	Thales/Utimaco HSMs, threshold-split across Election Commission staff
Casting	Ballot Casting Service	Validates token + signature, submits transaction to Fabric	Go/Node.js, stateless, horizontally scaled
Casting	Mixnet / Anonymization Relay	Decorrelates submission network path from registration	Nym or custom onion-routing layer
Ledger	Fabric Peers (per org)	Endorse, validate, and store ballot transactions	Hyperledger Fabric 2.5+/3.x, on-prem per org
Ledger	Fabric Ordering Service	BFT consensus on transaction order	Fabric BFT orderer (Raft as fallback)
Ledger	Chaincode	Ballot validation, double-spend (spent-token) check, event emission	Go chaincode
Tally	Tally Service	Homomorphic aggregation of encrypted ballots per channel	Rust/Go, using e.g. ElGamal/Paillier homomorphic scheme
Tally	Threshold Decryption Coordinator	Orchestrates multi-party decryption ceremony	Custom service + HSM integration, run live/observed
Aggregation	National Results Aggregator	Combines per-channel/per-province tallies into national result	Go service, reads finalized chaincode events only

Layer	Component	Responsibility	Suggested stack
Audit	Public Verifier Portal	Lets any voter/observer independently verify ledger + tally	Public read-only web app, static hosting + public API
Ops	Monitoring & SIEM	Node health, anomaly detection, security event correlation	Prometheus + Grafana, ELK/SIEM (e.g., Splunk or open-source equivalent)
Ops	Key Ceremony & HSM Management	Governs generation/rotation/custody of all cryptographic keys	Documented multi-party ceremony, hardware-backed

4. Data flow (end to end)

Registration (pre-election window): Voter authenticates via national eID → Identity Service checks roll → HSM-backed blind signature issued → voter stores unblinded token locally (encrypted on-device).
Casting (election day): Voter opens app → composes ballot → ballot encrypted client-side (homomorphic-compatible scheme) → submitted with token via mixnet/anonymization relay → Casting Service verifies token signature and checks it's unspent → transaction submitted to the relevant provincial Fabric channel.
Endorsement & ordering: Peers from a required subset of organizations (endorsement policy, e.g. 4-of-6 orgs) simulate and endorse the chaincode transaction → BFT ordering service sequences it into a block → block distributed to all peers on that channel → chaincode marks token as spent.
Voter receipt: Casting Service returns a cryptographic receipt (not revealing vote choice) letting the voter later confirm inclusion on the public ledger.
Close of polls: Chaincode freezes further submissions on each channel.
Tally: Tally Service homomorphically sums encrypted ballots per channel → Threshold Decryption Coordinator runs a live, observed ceremony requiring a quorum (e.g., 5-of-9) of key-share holders → per-channel results published with zero-knowledge proofs of correct decryption.
Aggregation: National Aggregator sums published per-channel results into the national result.
Public audit: Anyone can pull the full encrypted ledger + proofs from the Public Verifier Portal and independently recompute the tally; voters check their own receipt against the ledger.

5. Cryptography stack

Purpose	Primitive	Notes
Ballot secrecy in transit/storage	Additively homomorphic encryption (ElGamal or Paillier)	Enables tallying without individual decryption
Eligibility credential	Blind RSA or BBS+ blind signatures	Issuing authority never sees the unblinded token

Purpose	Primitive	Notes
Ledger integrity	Fabric's built-in transaction hashing + endorsement signatures	Standard PKI, per-org CAs
Threshold decryption of tally	Shamir's Secret Sharing over the decryption key, (k of n)	n = number of consortium orgs, k = quorum e.g. 5 of 9
Zero-knowledge proofs	zk-SNARKs (Groth16 or Plonk) for "correct decryption" and optionally "well-formed ballot" proofs	Adds public verifiability without revealing plaintext
Transport security	TLS 1.3 everywhere, mutual TLS between Fabric peers	Standard
Key custody	HSM-backed, split-knowledge, dual-control procedures	No single administrator can extract a usable key

6. Infrastructure & deployment topology (hybrid)

On-premises (per consortium org): Fabric peer nodes, ordering service nodes, HSMs, CA servers. This is non-negotiable for trust — no organization should run its ledger node or key share on infrastructure another party controls.
Cloud (Electoral Commission-managed or multi-cloud): API gateway, voter-facing web/app backends, Identity Service, mixnet relays, monitoring stack, public verifier portal. These are stateless or hold only encrypted/replicated data, so cloud elasticity helps absorb election-day traffic spikes.
Network segmentation: cloud-facing services sit in a DMZ; connections into the Fabric network from cloud services go through a tightly controlled, logged, rate-limited gateway — cloud compromise should never give direct write access to ledger peers.
Kubernetes for the cloud-hosted stateless services (auto-scaling for election-day load); Fabric nodes typically run on dedicated VMs/bare metal per org rather than shared/elastic infrastructure, for custody and auditability reasons.
Multi-region redundancy for cloud services; each on-prem org should have its own DR site.

7. Non-functional requirements

Requirement	Target	Design lever
Peak throughput	Tens of millions of votes across a multi-hour voting window; design for several thousand TPS at peak	Channel sharding by province, horizontal scaling of Casting Service

Requirement	Target	Design lever
Availability during voting window	99.99%+	Multi-region cloud, redundant on-prem nodes, no single point of failure in endorsement policy
Latency (vote submission to receipt)	Under 5 seconds at p95	Async endorsement pipeline, CDN edge termination
Auditability	100% of ballots independently verifiable post-election	Public ledger export, ZK proofs, published verifier tooling
Key compromise tolerance	Survive compromise of any single org's key share	Threshold (k-of-n) schemes throughout

8a. Threat model

A security reviewer's first question is "what are you actually defending against?" — here's the mapping from threat to the specific mitigation already in the design (or added below), so the design is auditable against the threat list rather than just described in the abstract.

Threat	What it looks like	Primary mitigation
Nation-state attacker	Well-resourced attempt to compromise a layer, subvert a consortium org, or manipulate results at scale	No single org can decrypt, order, or finalize alone (k-of-n threshold everywhere); independent orgs run their own peers on their own infrastructure; public verifiability means manipulation is detectable even if not preventable in real time
Insider threat (election staff, developers, admins)	Someone with legitimate access abuses it — stuffs ballots, leaks keys, alters chaincode	Dual control / two-person rule on all key operations; separation of duties between dev, ops, and key custodians; all admin actions logged to an append-only audit channel; reproducible builds so deployed code is independently verifiable against source
Coercion / vote buying	Someone pressures or pays a voter, then demands proof of how they voted	Receipt is *not* vote-revealing — it proves inclusion, not choice; support revoting (last-cast-vote-counts) so a coerced early vote can be silently overridden later; retain an in-person voting channel as an always-available uncoerced option
DDoS	Flooding the casting service or gateway during the voting window	CDN/WAF at the edge, elastic auto-scaling of stateless services, rate limiting per credential (not per IP, to avoid penalizing shared networks), poll-site fallback that doesn't depend on the public internet path
Device malware on voter's phone/PC	Malware that alters the vote before encryption, or exfiltrates the token	Cast-as-intended verification (Section 8c), key operations isolated in secure enclave/TEE where possible, independent verifier tooling outside the compromised device
Fake/spoofed voter apps	A malicious app impersonating the official voting app to harvest tokens or votes	Code signing with certificate pinning, published canonical app hashes voters can check out-of-band, public awareness campaign on official distribution channels only
Supply chain attacks	Compromised dependency, build pipeline, or HSM firmware	Signed releases, published SBOM (software bill of materials), reproducible builds, vetted/certified HSM vendors, dependency pinning with hash verification
Network-level deanonymization	Correlating a voter's identity-phase session with their casting-phase session via IP/timing	Mixnet/anonymization relay and enforced time delay between registration and casting (Section on Identity layer)
Consensus manipulation / node	A subset of orgs colluding to reorder, censor, or fork	BFT ordering requiring cross-org quorum; endorsement policy requires a majority of *independent* orgs, not just

Threat	What it looks like	Primary mitigation
collusion	the ledger	any majority of nodes
Chaincode / smart contract bugs	Logic error in ballot validation or double-spend checking	Mandatory independent security audit before each phase gate (Section 9); formal verification of chaincode where feasible; append-only ledger means any bug is visible and provable, not silently exploitable

8b. Operational governance & human processes

Large election systems fail more often from process gaps than from cryptographic breaks, so this needs to be as concrete as the technical design.

HSM / key ceremony procedure. Key generation happens in a scheduled, witnessed ceremony: representatives from every consortium org physically present (or verified via secure video for remote orgs), ceremony recorded, an independent notary/auditor present. The signing/decryption key is generated *inside* the HSM and split via Shamir's Secret Sharing into shares matching the consortium (Section 2); each share is written to a physically separate hardware token and handed to a named custodian from a different org, who stores it in their own physically secured location (safe, bank vault). No key ever exists whole outside the HSM. Any later use of the key (e.g., the tally decryption ceremony) requires a live quorum of custodians physically presenting their shares — this is deliberately slow and ceremonial by design, not an API call.

Node outage handling. The endorsement policy (e.g., 4-of-6 orgs) is chosen so voting continues uninterrupted if a minority of orgs' nodes go offline — this is the entire point of BFT-style quorums. An offline org's node resyncs from peers once it returns, since the ledger is fully replicated; it doesn't cause data loss, only a temporary loss of that org's own real-time visibility. Automated alerting notifies both that org's operators and the joint operations group when a node drops, with defined time-to-acknowledge and time-to-restore SLAs written into the consortium agreement, not just the tech stack.

No traditional rollback — an append-only ledger by design. Because the ledger is immutable, "rollback" isn't a technical operation here; the response to a discovered problem (e.g., a chaincode bug found mid-election) is: (1) freeze further submissions on the affected channel via a governance transaction requiring multi-org quorum sign-off — never a unilateral action, (2) do not alter or delete existing blocks, (3) deploy a corrected chaincode version for subsequent transactions, (4) publish a public incident report explaining exactly what happened and what was and wasn't affected. A pre-agreed, legally-grounded "election abort/extend" procedure should exist *before* election day, defining who can invoke it (requires cross-org quorum, not any single official) and what the fallback is (typically paper ballots at affected sites).

Incident response. A joint security operations center staffed across consortium orgs (not solely run by the Electoral Commission) operates for the full election period, with defined severity tiers, playbooks for the threats in Section 8a, and a joint incident-command structure so no single org can unilaterally characterize or respond to an incident — this mirrors the trust model of the rest of the system.

Dispute resolution. Because the ledger and tally proofs are public, disputes route through a defined legal/judicial channel that references the public verifier portal (Section 3) as evidence — any party can

independently recompute the tally from public data rather than relying on the Electoral Commission's word. This requires legal groundwork ahead of deployment: election law needs to explicitly define how cryptographic proofs are treated as evidence in electoral courts, and what a formal "recount" means when the underlying data is already public and immutable (typically: independent re-verification of the published proofs, not manual re-tallying).

8c. Client-side security

The voter's own device is usually the weakest link in any online voting system, so it deserves more than "we encrypt locally."

Device attestation: before allowing ballot submission, the app checks platform attestation (Play Integrity API on Android, App Attest/DeviceCheck on iOS) to detect a rooted/jailbroken device, a modified app binary, or an emulator — and blocks or warns accordingly.
Secure enclave / TEE usage: the actual cryptographic operations — generating the blinding factor, encrypting the ballot — should execute inside the device's hardware-backed secure enclave (Secure Enclave on iOS, StrongBox/TEE on Android) wherever the platform supports it, so malware running in the main OS can't directly read key material even if it can see the UI.
Cast-as-intended verification (Benaloh challenge): before final submission, let the voter (or an independent observer tool) "challenge" the encryption — the app reveals the randomness used to encrypt a *spoiled* test ballot (never the one actually cast) so an independent tool can confirm the encryption genuinely matches the selections shown on screen. This directly answers "how do I know the app encrypted what I actually chose" without ever exposing the real ballot's secrecy.
QR-based receipt confirmation: after casting, the app displays a QR code encoding the (non-vote-revealing) inclusion token/commitment, which the voter can scan with an independent verifier app or website to confirm their ballot reached the ledger — decoupling "did my vote count" verification from trusting the same app that cast it.
Independent, third-party verifier tooling: publish the verification protocol openly and encourage (or directly fund) civil-society or academic groups to build separate, independently-audited verifier apps — so voters aren't limited to trusting the official app's own self-report of what happened.
Open-source client + reproducible builds: publish the voting client's source code and provide reproducible build instructions with published canonical hashes, so security researchers and party observers can verify that what's distributed in app stores matches the audited source — closing the loop against both supply-chain tampering and fake-app spoofing (Section 8a).

9. Build phases

Phase 0 — Cryptographic core & threat model sign-off: finalize the blind-signature/credential scheme, homomorphic scheme, ZK proof system; independent security audit of the design before writing production code.
Phase 1 — Core ledger & chaincode: stand up Fabric consortium network (can start with 2–3 orgs in test), write and audit chaincode.
Phase 2 — Identity & casting services: integrate with national eID, build blind-signature issuance, casting pipeline, mixnet relay.
Phase 3 — Tally & audit tooling: threshold decryption ceremony tooling, public verifier portal.

Phase 4 — Pilot: run in a small real or shadow election, independent red-team and academic cryptographic review.
Phase 5 — Scale-out & hardening: load testing at national scale, chaos engineering on the on-prem/cloud boundary, full incident response runbooks.
Phase 6 — National deployment.

Independent security audits (ideally academic + commercial) should gate every phase transition — for a system this consequential, "it works in testing" is a necessary but nowhere-near-sufficient bar. The threat model (Section 8a) should be revisited and re-scored at every phase gate, since new components introduce new attack surface.